P.Vishwa nandan(192473015)

TASK A5 – DESIGN FOR CORRECTNESS AND SAFETY (4 MARKS)

1. Aim

To design safe database operations using JDBC transactions and PreparedStatement. The task ensures that money transfers remain correct even when a program fails and prevents SQL injection attacks during user authentication.

2. Problem Description

A hostel management application transfers ₹500 from a hostel wallet to a canteen wallet.

The application performs two database updates:

1. Deduct ₹500 from the hostel wallet.
2. Add ₹500 to the canteen wallet.

If the program fails between these two updates, money may be lost or incorrectly created.

The application also contains an unsafe login query that directly combines user input with an SQL statement.

The solution must use JDBC transactions and PreparedStatement to maintain data correctness and security.

3. Task A5.1 – Transaction Design (2 Marks)

3.1 What Is a Database Transaction?

A database transaction is a group of database operations treated as one logical unit.

All operations must either complete successfully or be cancelled together.

JDBC provides the following methods:

setAutoCommit(false)

Disables automatic commitment of individual SQL statements and allows multiple statements to be grouped into one transaction.

commit()

Permanently saves all successful operations in the current transaction.

rollback()

Cancels the changes made during the current transaction.

3.2 Wallet Transfer Design

The application transfers ₹500 from the hostel wallet to the canteen wallet.

Example starting balances:

Wallet	Initial Balance
Hostel Wallet	₹2,000
Canteen Wallet	₹1,000
Total	₹3,000

After a successful transfer:

Wallet	Final Balance
Hostel Wallet	₹1,500
Canteen Wallet	₹1,500
Total	₹3,000

The total money remains unchanged.

3.3 SQL Table Design

Create a wallet table in MySQL.

```
CREATE DATABASE IF NOT EXISTS hostel_wallet;

USE hostel_wallet;

CREATE TABLE wallets (
    wallet_id VARCHAR(30) PRIMARY KEY,
    balance DECIMAL(12,2) NOT NULL,
    CHECK (balance >= 0)
);

INSERT INTO wallets (wallet_id, balance)
```

VALUES

('HOSTEL', 2000.00),

('CANTEEN', 1000.00);

The wallet_id column uniquely identifies each wallet, and the balance column stores the available amount.

3.4 Transaction Algorithm Using SIMATS Standard Pseudocode

ALGORITHM: TRANSFER_MONEY

INPUT: Source Wallet ID, Destination Wallet ID, Transfer Amount = ₹500.

OUTPUT: Successful transfer message or transaction failure message.

BEGIN

1. OPEN a JDBC database connection.

2. STORE the current auto-commit setting.

3. SET auto-commit to FALSE.

4. BEGIN TRANSACTION.

5. PREPARE an SQL statement to deduct ₹500
   from the source wallet.

6. EXECUTE the deduction statement.

7. IF exactly one source wallet is not updated THEN

      RAISE "Insufficient balance or source wallet not found".

   END IF.

8. PREPARE an SQL statement to add ₹500
to the destination wallet.

9. EXECUTE the addition statement.

10. IF exactly one destination wallet is not updated THEN

RAISE "Destination wallet not found".

END IF.

11. COMMIT the transaction.

12. DISPLAY "Transfer completed successfully".

13. IF any database operation fails THEN

ROLLBACK the transaction.

DISPLAY "Transfer failed. No balance changes saved".

END IF.

14. RESTORE the original auto-commit setting.

15. CLOSE the database connection.

END.

3.5 Java Implementation – WalletTransfer.java

Explanation

Step 1 – Open Connection

The application establishes a connection to MySQL.

Step 2 – Disable Auto-Commit

con.setAutoCommit(false);

This allows both wallet updates to be managed as one transaction.

Step 3 – Deduct ₹500

The first UPDATE statement subtracts ₹500 from the hostel wallet.

The condition balance >= ? prevents the wallet from having an insufficient balance.

Step 4 – Add ₹500

The second UPDATE statement adds ₹500 to the canteen wallet.

Step 5 – Commit

con.commit();

Both updates become permanent only after successful execution.

Step 6 – Rollback

con.rollback();

If an SQL operation fails before the commit, the database cancels the changes made in the transaction.

Step 7 – Close Resources

Try-with-resources automatically closes the database connection and prepared statements.

3.6 What Happens If the Program Fails Between Updates?

Consider the following situation:

1. ₹500 is deducted from the hostel wallet.
2. The program suddenly fails before adding ₹500 to the canteen wallet.
3. The transaction has not yet been committed.

The database rolls back the transaction when the connection is terminated without a commit.

Therefore, the original balances are restored.

Wallet	Before Failure	After Rollback

Hostel	₹2,000	₹2,000
Canteen	₹1,000	₹1,000
Total	₹3,000	₹3,000

No money is lost or created.

Important: The transaction must be managed by a database that supports transactions, and both wallet tables must use a transactional storage engine such as InnoDB. If a network failure occurs during commit, the application may not know whether the commit succeeded; production systems should use transaction identifiers and reconciliation to handle uncertain outcomes.

4. Task A5.2 – Injection Analysis (2 Marks)

4.1 What Is SQL Injection?

SQL injection is a security attack in which a user enters specially crafted input that changes the meaning of an SQL query.

It occurs when user input is directly combined with an SQL statement.

The original login query is:

String sql =

"SELECT * FROM users WHERE id='" + id

+ "' AND pwd='" + pwd + "'";

This query is unsafe because the entered values are directly joined into the SQL statement.

4.2 How Can an Attacker Bypass Login?

Suppose the application expects:

User ID: admin

Password: admin123

However, an attacker enters the following input in the User ID field:

' OR '1'='1' --

The password field can contain any incorrect value.

For example:

User ID: ' OR '1'='1' --

Password: wrongpassword

The resulting SQL query becomes:

```
SELECT * FROM users
WHERE id=''
OR '1'='1' -- '
AND pwd='wrongpassword'
```

Why Does This Query Succeed?

1. The condition '1'='1' is always TRUE.
2. The OR operator makes the WHERE condition true.
3. The -- characters start an SQL comment in MySQL.
4. The remaining password condition is ignored.

Therefore, the query may return user records even though the attacker entered an invalid password.

The application may incorrectly accept the login.

4.3 Problems With the Original Login Query

1. User input is directly added to the SQL statement.
2. SQL conditions can be modified through crafted input.
3. Authentication can be bypassed.
4. Unauthorised users may access the application.
5. Sensitive information may be exposed.

4.4 Secure Solution Using PreparedStatement

PreparedStatement separates the SQL query from the values supplied by the user.

The SQL statement uses placeholders (?) instead of directly inserting input values.

Corrected SQL Query

```
SELECT * FROM users
WHERE id = ? AND pwd = ?;
```

Java Program – SecureLogin.java

Explanation

Step 1 – Prepare SQL Statement

```
String sql =
    "SELECT * FROM users WHERE id = ? AND pwd = ?";
```

The question marks represent parameter placeholders.

Step 2 – Set Parameters

ps.setString(1, id);

ps.setString(2, pwd);

The entered username and password are supplied as values rather than SQL commands.

Step 3 – Execute Query

ResultSet rs = ps.executeQuery();

The query is executed using the supplied parameter values.

Step 4 – Check Login

return rs.next();

If a matching record exists, the method returns true. Otherwise, it returns false.

4.5 Why Does PreparedStatement Prevent SQL Injection?

PreparedStatement treats user input as data rather than executable SQL syntax.

For example, the crafted input:

' OR '1'='1' --

is treated as the actual username value.

It cannot change the structure of the prepared SQL query.

Therefore, the authentication check rejects the invalid login unless the exact supplied values match a database record.

Important security improvement: The example demonstrates the required PreparedStatement solution using the assignment's pwd column. In a real authentication system, passwords should never be stored in plaintext. The database should store a salted password hash, and the application should verify the submitted password against that hash using an appropriate password-hashing algorithm.

5. Alternative Concept and Complexity Analysis

5.1 Alternative – Two Separate Auto-Commit Updates

Instead of using one transaction, the application could execute the hostel debit and canteen credit as two independent SQL statements.

By default, JDBC connections generally use auto-commit mode.

This means every successfully executed SQL statement is committed separately.

Example:

```
String debitSQL =
   "UPDATE wallets SET balance = balance - 500 " +
   "WHERE wallet_id = 'HOSTEL'";

String creditSQL =
   "UPDATE wallets SET balance = balance + 500 " +
   "WHERE wallet_id = 'CANTEEN'";

Statement st = con.createStatement();

st.executeUpdate(debitSQL);

// Program may fail here.

st.executeUpdate(creditSQL);
```

What Happens If the Program Fails Between the Updates?

The first debit statement is already committed.

If the program fails before executing the credit statement, the hostel wallet loses ₹500, but the canteen wallet does not receive it.

Example:

Wallet	Initial Balance	After Failure
Hostel	₹2,000	₹1,500
Canteen	₹1,000	₹1,000
Total	₹3,000	₹2,500

Therefore, ₹500 is lost from the combined wallet balances.

This violates the requirement that money must never be lost or created.

5.2 Transaction vs Auto-Commit Comparison

Feature	One JDBC Transaction	Two Auto-Commit Updates
Number of updates	Two	Two
Commit operations	One final commit	One commit per statement
Rollback support	Both updates can be cancelled together	Earlier committed update cannot be automatically undone
Failure between updates	Original balances restored	First update remains committed
Data correctness	Maintains all-or-nothing behaviour	Partial update is possible
Lock management	Resources held until transaction ends	Each statement commits separately
Suitability	Recommended	Not suitable for money transfers

5.3 Time Complexity Analysis

Let n represent the number of wallet records.

Both approaches update two wallet records using their primary keys.

Transaction Approach

The application performs:

1. One debit update.
2. One credit update.
3. One commit operation.

The wallet updates have constant-time execution with respect to the number of records when the wallet IDs use primary-key indexes, excluding database I/O, locking, and transaction-log overhead.

Approximate data-operation complexity:

O(1)

The transaction adds coordination and commit overhead.

Auto-Commit Approach

The application performs:

1. One debit update with an automatic commit.
2. One credit update with another automatic commit.

Approximate data-operation complexity:

O(1)

However, each statement has its own commit overhead.

The asymptotic complexity is similar, but the correctness guarantee is weaker.

5.4 Time and Space Comparison

Factor	Transaction	Separate Auto-Commit
Data update complexity	O(1) per indexed update	O(1) per indexed update
Commit operations	One final commit	Two independent commits
Transaction overhead	Requires transaction coordination	No shared transaction
Extra Java memory	O(1) for fixed transfer	O(1) for fixed transfer
Failure safety	Both updates roll back together	Partial updates possible

The exact execution time depends on the database, storage engine, network, and transaction-log settings.

5.5 Engineering Decision

A JDBC transaction is strongly recommended for wallet transfers because both operations must succeed together.

The small additional transaction overhead is justified by the protection it provides against partial updates.

PreparedStatement is also recommended for login verification because it separates SQL commands from user input and prevents SQL injection through parameter values.

6. Testing and Verification

The following test cases verify the implementation.

6.1 Transaction Test Cases

Test Case	Action	Expected Result
TC1	Transfer ₹500 with sufficient balance	Both wallets updated
TC2	Hostel wallet has less than ₹500	Transfer rejected
TC3	Hostel wallet does not exist	Transaction rolled back
TC4	Canteen wallet does not exist	Hostel debit rolled back

TC5	Database operation fails before commit	All transaction changes rolled back
TC6	Successful transfer	Total wallet balance remains unchanged

6.2 SQL Injection Test Cases

Test Case	User ID	Password	Expected Result
TC1	Valid user ID	Correct password	Login accepted
TC2	Valid user ID	Incorrect password	Login rejected
TC3	Invalid user ID	Any password	Login rejected
TC4	' OR '1'='1' --	wrongpassword	PreparedStatement rejects login
TC5	Empty user ID	Empty password	Login rejected

6.3 Expected Output – Successful Transaction

₹500 transferred successfully.

6.4 Expected Output – SQL Injection Attempt

Invalid username or password.

The actual database outcome must be verified using SELECT queries after the transaction and login tests.

7. Final Result

The hostel-to-canteen wallet transfer was designed using JDBC transaction management.

The application disables auto-commit, performs both wallet updates, and commits only after successful execution. If an operation fails, rollback restores the original wallet balances.

The SQL injection vulnerability in the original login query was identified and corrected using PreparedStatement.

Finally, transaction-based updates were compared with separate auto-commit statements using execution cost and data-correctness considerations.

Thus, the task demonstrates safe database transactions, rollback handling, and protection against SQL injection in Java JDBC applications.